

ENGENHARIA DA QUALIDADE E CONFIABILIDADE

Arquiteturas tolerantes a defeitos: Sistemas de proteção, Arquitetura de automonitoramento, Programação N-versões (N-version programming) e Diversidade de software

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



1 Arquiteturas Tolerantes a Defeitos

- Conhecendo o Hadoop
- Hadoop File System (HDFS)
- Redundância no Hadoop
- Ecossistema Hadoop
- Sistemas de Proteção
- Detecção de falhas no Hadoop
- Arquitetura de Automonitoramento
- Diversidade de Hardware
- Diversidade de Software
- Programação N-versões

2 Conclusão

Diversos pesquisadores investigaram formas de tornar sistemas confiáveis mesmo na presença de defeitos:

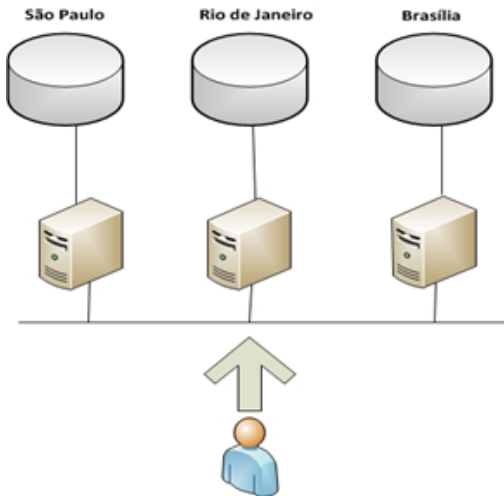
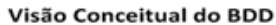
- **Orres-Pomales (2000):** levantou várias técnicas de tolerância a defeitos em software.
- **Pullum (2001):** descreveu arquiteturas de sistemas tolerantes a defeitos.

Nesta seção serão apresentados três padrões arquiteturais clássicos utilizados em sistemas críticos.

Objetivo

Permitir que o sistema continue operando mesmo quando componentes falham.

Sistema distribuido





O que é

Hadoop é um framework de computação distribuída projetado para armazenar e processar grandes volumes de dados em clusters de computadores comuns (commodity hardware), garantindo escalabilidade horizontal e tolerância a falhas.

Ideia principal

Ao invés de usar um supercomputador, o Hadoop divide o problema em várias partes menores e executa cada parte em diferentes máquinas ao mesmo tempo. Se uma máquina falhar, outra assume automaticamente sua função.

Componentes fundamentais

- HDFS — sistema de arquivos distribuído e replicado
- MapReduce — modelo de processamento paralelo
- YARN — gerenciador de recursos do cluster

Como funciona

O Hadoop não possui um terminal próprio. Utilizamos o terminal Linux (bash) e executamos comandos através do cliente HDFS: `hdfs dfs`

Primeiro comando

```
hdfs dfs -ls /
```

Se o comando listar diretórios, o sistema de arquivos distribuído já está acessível.

Interpretação

A partir desse ponto, todos os comandos executados com `hdfs dfs` manipulam arquivos armazenados no cluster, não no computador local.

```
hdfs dfs -ls /
```

```
hdfs dfs -mkdir /dados
```

```
\begin{verbatim}
```

```
hdfs dfs -put arquivo.txt /dados
```

```
hdfs dfs -cat /dados/arquivo.txt
```

```
hdfs dfs -get /dados/arquivo.txt
```

Comparação: Linux vs HDFS

Equivalência de comandos

Linux	HDFS
ls	hdfs dfs -ls
mkdir	hdfs dfs -mkdir
cp	hdfs dfs -put / -get
cat	hdfs dfs -cat

Importante

Os comandos parecem Linux, mas operam em máquinas do cluster. Os dados não estão no computador local.

Conclusão

O HDFS funciona como um sistema de arquivos remoto acessado via terminal.

Ideia central

O Hadoop foi projetado assumindo que falhas de hardware são normais. Em vez de evitar falhas, o sistema tolera falhas automaticamente.

- Usa armazenamento distribuído (HDFS)
- Dados são replicados entre máquinas
- O sistema continua funcionando mesmo com nós quebrados

Exemplo de Replicação

Bloco	Nó A	Nó B	Nó C
B1	cópia	cópia	cópia
B2	cópia	cópia	cópia
B3	cópia	cópia	cópia

Resultado

Se um computador falhar, os dados continuam acessíveis nos outros.



Hadoop Ecosystem

oozie

(Work flow)

HCatalog

Table & schema
Management



Pig

(Scripting)



Hive

(Sql Query)



(Machine
Learning)



Drill
(Interactive
Analysis)



AVRO
(JSON)

Thrift

(Cross
Language
Service)

APACHE
HBASE

HBASE
(Columnar
Store)



Sqoop

(Data Collection)



Zookeeper
(Coordination)



Ambari

Apache Ambari
(Management
& Monitoring)

Mapreduce
(Data Processing)



Yarn
(Cluster Resource Management)

HDFS

(Hadoop Distributed File system)



Um **sistema de proteção** é um sistema especializado associado a outro sistema principal.

- Atua como supervisor independente;
- Observa o ambiente e o comportamento do sistema controlado;
- Intervém quando o sistema principal não reage corretamente.

Aplicações típicas

- Controle industrial (plantas químicas)
- Equipamentos críticos
- Trens autônomos

Exemplo: Trem Autônomo

Situação: o trem ultrapassa sinal vermelho.

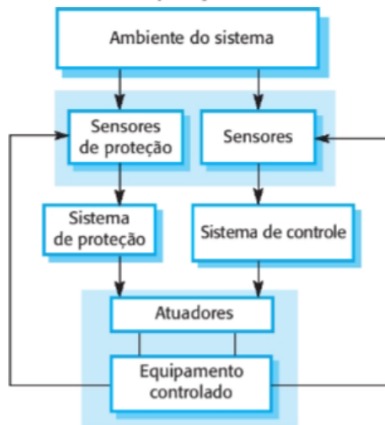
- ❶ O sistema principal deveria desacelerar
- ❷ Se isso não ocorrer
- ❸ O sistema de proteção assume o controle
- ❹ Freios são acionados automaticamente

Importante

O sistema de proteção não depende da decisão do sistema principal. Ele atua mesmo se o software de controle estiver defeituoso.

Visão esquemática de um sistema de proteção

FIGURA 11.6 Arquitetura do sistema de proteção.



O sistema monitora dois elementos:

- O comportamento do sistema controlado
- O ambiente físico ao redor

Ação de Proteção

Se for detectada uma situação perigosa:

- O processo é desligado
- Ou mecanismos de segurança são acionados

Exemplo: abertura de válvula de alívio de pressão

Arquiteturas de proteção utilizam redundância física:

- Dois conjuntos de sensores
 - Sensores de operação normal
 - Sensores dedicados à segurança
- Sensores de backup em caso de falha
- Atuadores redundantes

Motivação

O sistema de proteção deve continuar operando mesmo se o próprio sistema monitorado estiver falhando.

O sistema principal tenta operar corretamente

O sistema de proteção garante que nada catastrófico aconteça

Ele não evita defeitos — ele evita acidentes.

Um sistema de proteção implementa apenas a funcionalidade crítica necessária para mover o sistema de um estado perigoso para um estado seguro.

- Não executa operação normal
- Atua somente em emergência
- Pode desligar completamente o sistema

Arquitetura Geral

Sistema principal complexo + sistema de backup simples e independente

Exemplo: Sistema de Backup do Ônibus Espacial

O software de controle do ônibus espacial possuía um sistema secundário com funcionalidade limitada:

- Não realizava todas as operações de navegação
- Não executava todas as rotinas de controle
- Apenas a função essencial: **"levá-lo para casa"**

Objetivo

Caso o sistema principal falhasse, o sistema de proteção conseguiria realizar o pouso com segurança.

Como o sistema é pequeno e simples:

- É possível validar melhor a especificação
- Verificar formalmente o comportamento
- Investir mais na prevenção e detecção de defeitos

Meta de Engenharia

Probabilidade muito baixa de falha sob demanda:

$$P(\text{falha} \mid \text{demanda}) \approx 0,001$$

Demandas são raras $\Rightarrow$ falhas extremamente raras



Heartbeat

Cada máquina envia sinais periódicos ao servidor principal.

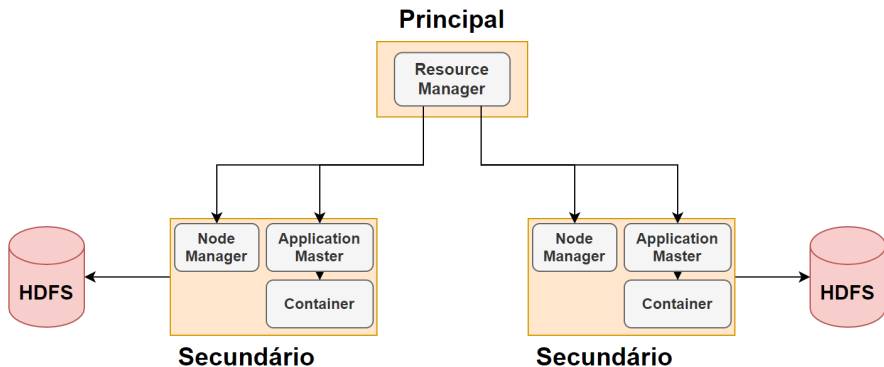
Se parar de responder:

- 1 O sistema detecta falha
- 2 Identifica blocos perdidos
- 3 Cria novas cópias automaticamente

Importante

O cluster se auto-repara sem intervenção humana.

Arquitetura de nós hadoop



O próprio sistema monitora sua operação e detecta falhas internas.

- Cálculos executados em canais independentes
- Resultados são comparados
- Divergência indica defeito

Princípio

Se dois sistemas iguais recebem a mesma entrada, devem produzir a mesma saída.

Processo de funcionamento:

- 1 Entrada é enviada para múltiplos canais
- 2 Cada canal calcula independentemente
- 3 Resultados são comparados

Resultado

- Iguais → operação considerada correta
- Diferentes → falha detectada

O sistema então sinaliza erro e transfere o controle.

Para evitar falhas comuns, os canais não devem ser idênticos.

- Processadores diferentes
- Chipsets de fabricantes distintos
- Implementações físicas independentes

Motivação

Reduzir a probabilidade de defeitos de projeto afetarem todos os canais ao mesmo tempo.

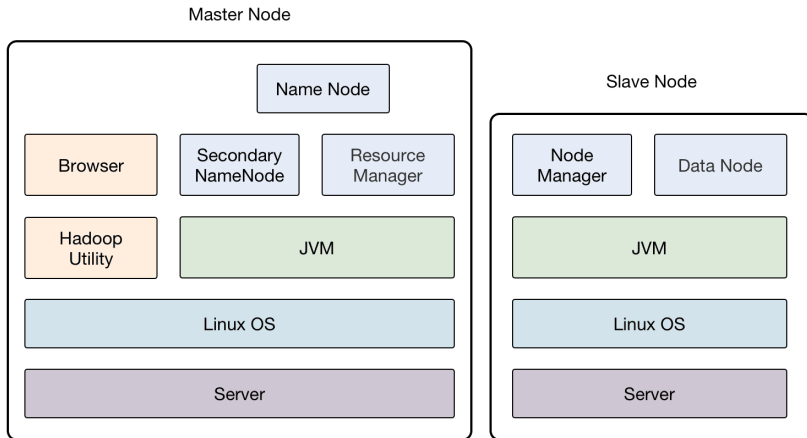
Também é necessário evitar falhas simultâneas de software.

- Implementações independentes
- Equipes diferentes
- Linguagens de programação diferentes

Problema evitado

O mesmo bug não deve existir em todos os canais.

Nó master e slave no hadoop



A arquitetura pode ser usada quando a confiabilidade é mais importante que disponibilidade.

- Se houver divergência → sistema desliga

Exemplo

Sistemas médicos de diagnóstico: uma resposta errada é pior que parar o sistema.

Quando o sistema não pode parar:

- Vários sistemas automonitorados em paralelo
- Unidade de chaveamento seleciona saída válida

Objetivo

Manter operação mesmo com falhas internas.

Exemplo: Controle de Voo

Sistema de controle de voo usa múltiplos computadores:

- Computações paralelas com mesma entrada
- Filtros detectam saída defeituosa
- Saída alternativa é selecionada

O sistema pode continuar operando mesmo com múltiplas falhas.

Diversidade aplicada de várias formas:

- 1 Processadores diferentes
- 2 Fornecedores distintos
- 3 Software secundário mais simples
- 4 Equipes independentes
- 5 Linguagens de programação diferentes

O sistema não tenta evitar falhas

Ele detecta falhas imediatamente

A segurança vem da comparação entre múltiplas computações independentes.

A programação N-versões é uma técnica de tolerância a defeitos baseada em redundância de software.

- Várias implementações independentes do mesmo sistema
- Executadas em paralelo
- Resultados comparados por votação

Objetivo

Garantir saída correta mesmo se uma versão falhar.

A ideia foi inspirada na tolerância a falhas de hardware:

- Triple Modular Redundancy (TMR)
- Três unidades executam o mesmo cálculo
- Um votador escolhe o resultado majoritário

Comportamento

Se uma unidade falhar, sua saída é ignorada.

O comparador recebe múltiplas saídas:

$$saida = maioria(saida_1, saida_2, saida_3)$$

- 2 iguais → resultado aceito
- 1 diferente → considerado defeituoso

Reconfiguração

O sistema pode remover automaticamente a unidade defeituosa e continuar operando.

A técnica assume:

- Falhas ocorrem independentemente
- Diferentes implementações não possuem o mesmo erro

Falha de modo comum

Se todas as versões tiverem o mesmo defeito, o sistema produzirá uma resposta errada consistente.

Extensão do conceito de TMR para software:

- N implementações independentes
- Desenvolvidas por equipes diferentes
- Linguagens e algoritmos diferentes

Saídas incoerentes ou atrasadas são descartadas.

Para tolerar uma falha:

$$N \geq 3$$

- Uma versão pode falhar
- Duas restantes definem o resultado correto

Condição

Pelo menos duas respostas coerentes devem existir.

Utilizado em sistemas críticos de segurança:

- Sinalização ferroviária
- Controle aeronáutico
- Proteção de reatores nucleares

A falha do sistema pode causar consequências catastróficas.

Vantagem:

- Alta disponibilidade

Desvantagem:

- Múltiplas equipes de desenvolvimento
- Alto custo de implementação

Uso típico

Aplicado apenas quando não é possível depender de um sistema de proteção simples.

Não confiar em uma implementação

Confiar no consenso entre implementações independentes

A segurança vem da diversidade.

Todas as arquiteturas tolerantes a defeitos dependem da diversidade de software.

Baseia-se no pressuposto de que implementações diferentes da mesma especificação:

- São independentes
- Não possuem erros em comum
- Não falham simultaneamente

Objetivo: aumentar a confiabilidade do sistema.

O software deve ser desenvolvido por times distintos:

- Sem comunicação entre si
- Interpretações independentes da especificação
- Redução de erros comuns

Isso reduz mal-entendidos coletivos da especificação.

A organização pode exigir diferenças explícitas entre versões.

- 1 Diferentes métodos de projeto
- 2 Diferentes linguagens de programação
- 3 Diferentes ferramentas de desenvolvimento
- 4 Diferentes algoritmos

Métodos de projeto

- Orientado a Objetos
- Orientado a Funções

Linguagens

- Ada
- C++
- Java

Ferramentas

- Compiladores distintos
- IDEs diferentes

Pode-se exigir algoritmos diferentes para o mesmo problema.

Vantagem:

- Redução de falhas simultâneas

Desvantagem:

- Dificuldade em cumprir requisitos de desempenho
- Restrição à liberdade de projeto

Se versões forem independentes:

$$Confiabilidade_{global} = Confiabilidade_1 \times Confiabilidade_2 \times Confiabilidade_3$$

Exemplo:

POFOD de cada canal = 0,001

Sistema com três canais:

$$0,001^3 = 0,000000001$$

Sistema torna-se um milhão de vezes mais confiável.

Independência completa não ocorre na prática.

Estudos mostram que equipes independentes:

- Cometem erros semelhantes
- Interpretam errado as mesmas partes da especificação

- 1 Mesma formação acadêmica
- 2 Mesmos livros e técnicas aprendidas
- 3 Dificuldade comum com especialistas do domínio

Se os requisitos estiverem errados:

- Todas as versões estarão erradas

Se a especificação for ambígua:

- Todos podem interpretar da mesma forma equivocada

Criar especificações independentes em formatos diferentes:

- Especificação formal
- Modelo baseado em estados
- Linguagem natural

Limitação: não resolve erros de requisitos.

Estudos mostram:

Sistema de três canais é de 5 a 9 vezes mais confiável que sistema único.

Abordagens N-versões geralmente aumentam a confiabilidade.

Nem sempre compensa:

- Alto custo de desenvolvimento
- Sistema único bem projetado pode ser suficiente

Recomendado principalmente para:

- Sistemas críticos de segurança
- Sistemas de missão

Pode ser suficiente:

- Sistema principal
- Backup simples com funções limitadas

Até o sistema principal ser reiniciado.

Diversidade de software:

- Aumenta confiabilidade
- Não garante independência total
- Possui alto custo
- Indicado para sistemas críticos